

Amostra de Dicionário Inglês CSV

https://gemini.google.com/app/a1b00da2bfb7da3a

User prompt: Existe algum dicionário/biblioteca neste padrão para download online que eu possa usar?

Response: Sim! Existem diversos repositórios públicos, datasets e bibliotecas de código aberto que contêm o dicionário em inglês estruturado exatamente com palavras e *part-of-speech* (POS).

Abaixo estão as principais opções prontas para download e como você pode baixá-las ou usá-las:

1. Kaggle: OPTED (Online Plain Text English Dictionary)

O OPTED é uma versão em formato legível e em CSV baseada no dicionário Webster's Unabridged Dictionary.

- O que possui: Colunas com a Palavra (Word), Classe Gramatical (Part of Speech) e Definição (Definition).

Como baixar:

- Acesse o Kaggle e procure por "The Online Plain Text English Dictionary (OPTED)".
- Faça o download direto do arquivo .csv (aproximadamente 100.000+ palavras).

2. Repositórios no GitHub (Arquivos .csv e .json Prontos)

No GitHub existem repositórios focados em disponibilizar datasets de palavras com POS:

- lorenza12/English-Words-Definitions-and-Parts-of-Speech
 - Contém um arquivo .csv pré-construído com palavras, definições e suas respectivas classes gramaticais (Noun, Verb, Adjective, etc.).
- Maximax67/Words-CEFR-Dataset
 - Um dataset moderno em .csv / SQLite que inclui palavras em inglês, suas classes gramaticais (POS tags), frequência de uso e níveis de proficiência (A1-C2).

3. SCOWL / ESDB (English Speller Database)

- Website: wordlist.aspell.net
- É a base de dados oficial usada por seletores de ortografia como Aspell e Hunspell no Linux e navegadores.
- Eles disponibilizam pacotes de listas de palavras (wordlists) classificadas por categorias gramaticais (substantivos, verbos, adjetivos) para download direto em .txt.

4. Gerar o seu próprio CSV completo via Python (Recomendado)

Se você precisa de um CSV limpo apenas com Word e Part-of-Speech atualizados (com base na base oficial WordNet), você pode gerar o arquivo localmente no seu computador em poucos segundos usando Python e a biblioteca NLTK:

Passo 1: Instale o NLTK

No seu terminal:

```
pip install nltk
```

Passo 2: Execute este script Python

Copie e rode o código abaixo. Ele processará todas as palavras do dicionário WordNet e criará um arquivo chamado `dicionario_ingles_pos.csv`:

```
import csv
import nltk
from nltk.corpus import wordnet as wn

# Baixa o banco de dados do WordNet se ainda não tiver baixado
nltk.download('wordnet')

# Mapeamento das siglas do WordNet para nomes amigáveis
POS_MAP = {
    'n': 'Noun',
    'v': 'Verb',
    'a': 'Adjective',
    's': 'Adjective', # Adjetivo satélite
    'r': 'Adverb'
}

words_pos = {}

# Percorre todas as synsets do WordNet
for synset in wn.all_synsets():
    pos = POS_MAP.get(synset.pos(), 'Other')
    for lemma in synset.lemmas():
        word = lemma.name().lower().replace('_', ' ')
        # Salva a palavra e sua classe gramatical (evitando duplicatas exatas)
        if word not in words_pos:
            words_pos[word] = set()
            words_pos[word].add(pos)

# Ordena as palavras em ordem alfabética
sorted_words = sorted(words_pos.keys())

# Escreve o arquivo CSV
with open('dicionario_ingles_pos.csv', mode='w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['Word', 'Part of Speech'])

    for word in sorted_words:
        pos_list = "/".join(sorted(list(words_pos[word])))
        writer.writerow([word, pos_list])

print("Arquivo 'dicionario_ingles_pos.csv' gerado com sucesso!")
```

Esse script gera um arquivo com mais de **140.000 palavras únicas**, organizadas em ordem alfabética e acompanhadas por suas classes gramaticais.

User prompt: Gere para mim um programa em python que analise um texto contido em um arquivo .csv estruturado com colunas article_id,age,case_id,case_text,gender (essa classificação está escrita na primeira linha, então é melhor pular ela ao fazer a leitura) e crie um arquivo similar, mas normalizado (quero normalizar números com unidades de medida, onde no texto original eles podem ou não conter um espaço entre o número e a unidade de medida; a normalização também deve quebrar palavras tipo haven't, aren't, wouldn't, you're, etc e colocá-las de forma have not, are not, etc). Este arquivo deve ser salvo como Dados > casesNormalizado.csv. A partir do arquivo Dados > casesNormalizado.csv o programa dele ler ele e criar um novo arquivo Dados > partOfSpeech.csv e para cada palavra encontrada em Dados > casesNormalizado.csv ele deve por qual part-of-speech. O arquivo com o texto se encontra em Dados > cases.csv e a biblioteca com cada palavra em Dados > dicionario_ingles.csv. Use Regex, considere que no texto podem ser encontrados números com unidades de medida com ou sem espaço entre o número e a unidade de medida; palavras não encontradas na biblioteca devem ser classificados como "Medico", numeros devem ser classificados como "Number" e números com unidade de medida como "ExamResult". Considere também que o código será rodado em ambiente Linux.

Response: Este script Python realiza a normalização de textos médicos e a classificação de Part-of-Speech (POS) utilizando expressões regulares e manipulação nativa de CSV, totalmente compatível com Linux.

Código Python (processador_casos.py)

```
import csv
import os
import re
from pathlib import Path

# Estrutura de diretórios no Linux
```

```
DATA_DIR = Path("Dados")
CASES_FILE = DATA_DIR / "cases.csv"
NORMALIZED_FILE = DATA_DIR / "casesNormalizado.csv"
DICT_FILE = DATA_DIR / "dicionario_ingles.csv"
POS_FILE = DATA_DIR / "partOfSpeech.csv"

# Regex para unidades de medida médicas comuns
UNITS_REGEX = r"(?:mg|g|kg|ml|l|mcl|mmol/L|mg/dl|cm|mm|mmhg|bpm|°C|°F|u/l|iu/meq|%)%"

# Regras de expansão de contrações em inglês
CONTRACTIONS = [
    (re.compile(r"\bcan't\b", re.IGNORECASE), "can not"),
    (re.compile(r"\bwon't\b", re.IGNORECASE), "will not"),
    (re.compile(r"\bshan't\b", re.IGNORECASE), "shall not"),
    (re.compile(r"(\b\w+)n't\b", re.IGNORECASE), r"\1 not"),
    (re.compile(r"(\b\w+)re\b", re.IGNORECASE), r"\1 are"),
    (re.compile(r"(\b\w+)ve\b", re.IGNORECASE), r"\1 have"),
    (re.compile(r"(\b\w+)ll\b", re.IGNORECASE), r"\1 will"),
    (re.compile(r"(\b\w+)m\b", re.IGNORECASE), r"\1 am"),
    (re.compile(r"(\b\w+)d\b", re.IGNORECASE), r"\1 would"),
    (
        re.compile(
            r"(\bit|he|she|that|there|what|where|who)'s\b", re.IGNORECASE
        ),
        r"\1 is",
    ),
]

def normalize_text(text: str) -> str:
    """Aplica expansão de contrações e padronização de espaços em unidades de medida."""
    for pattern, replacement in CONTRACTIONS:
        text = pattern.sub(replacement, text)

    # Normaliza números e unidades garantindo 1 espaço de separação
    unit_pattern = re.compile(
        rf"(\b\d+(?:\.\d+)?)\s*({UNITS_REGEX})\b", re.IGNORECASE
    )
    text = unit_pattern.sub(r"\1 \2", text)
    return text

def step_1_normalize():
    """Lê Dados/cases.csv, ignora o cabeçalho original e gera Dados/casesNormalizado.csv."""
    os.makedirs(DATA_DIR, exist_ok=True)

    with open(CASES_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        header = next(reader) # Pula o cabeçalho original

        rows = []
        for row in reader:
            if not row:
                continue
            article_id, age, case_id, case_text, gender = row
            normalized_text = normalize_text(case_text)
            rows.append([article_id, age, case_id, normalized_text, gender])

    with open(
        NORMALIZED_FILE, mode="w", encoding="utf-8", newline=""
    ) as outfile:
        writer = csv.writer(outfile)
        writer.writerow(header)
        writer.writerows(rows)

def load_dictionary() -> dict:
    """Carrega o dicionário de inglês no formato {palavra: pos}."""
    dict_pos = {}
    if not DICT_FILE.exists():
        return dict_pos

    with open(DICT_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        next(reader, None) # Pula o cabeçalho do dicionário
        for row in reader:
            if len(row) >= 2:
                word, pos = row[0].strip().lower(), row[1].strip()
                dict_pos[word] = pos
    return dict_pos

def step_2_classify_pos():
    """Lê Dados/casesNormalizado.csv e gera o mapeamento POS em Dados/partOfSpeech.csv."""
    dict_pos = load_dictionary()

    exam_result_pattern = re.compile(
        rf"^\d+(?:\.\d+)?\s*({UNITS_REGEX})$", re.IGNORECASE
    )
    number_pattern = re.compile(r"^\d+(?:\.\d+)?$")
    token_extractor = re.compile(
        rf"(\b\d+(?:\.\d+)?\s*({UNITS_REGEX})\b|\b\w+\b", re.IGNORECASE
    )

    pos_output = []

    with open(NORMALIZED_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        next(reader) # Pula o cabeçalho

        for row in reader:
            if not row:
                continue
            article_id, _, case_id, case_text, _ = row
            tokens = token_extractor.findall(case_text)

            for token in tokens:
                token_clean = token.strip()

                if exam_result_pattern.match(token_clean):
                    pos = "ExamResult"
                elif number_pattern.match(token_clean):
                    pos = "Number"
                else:
                    word_lower = token_clean.lower()
                    pos = dict_pos.get(word_lower, "Medico")

                pos_output.append([article_id, case_id, token_clean, pos])

    with open(POS_FILE, mode="w", encoding="utf-8", newline="") as outfile:
        writer = csv.writer(outfile)
        writer.writerow(["article_id", "case_id", "word", "pos"])
        writer.writerows(pos_output)
```

```
if __name__ == "__main__":
    step_1_normalize()
    step_2_classify_pos()
    print("Processamento concluído com sucesso!")
```

Como executar no Linux

1. Salve o script na raiz do projeto onde a pasta Dados se encontra.
2. Execute o comando no terminal:

```
python3 processador_casos.py
```

User prompt: Em anexo está o arquivo cases.csv. O que quis dizer na parte de normalizar os números com unidades de medida é que eu quero que o número e a unidade de medida não sejam separados. Outro problema, no arquivo partOfSpeech.csv gerado cada palavra está em uma linha diferente; eu só quero que ele mude de linha quando for para outro caso. Não é necessário que a palavra sendo referenciada esteja neste arquivo, apenas que a palavra na posição i do texto normalizado tenha sua part-of-speech na posição i em partOfSpeech.csv. Também na normalização, considere que algumas palavras podem ou não iniciar com letras maiúsculas e que o dicionário só contém palavras iniciadas com letras minúsculas. Por favor altere o código a seguir levando estes pontos em consideração. import csv import os import re from pathlib import Path # Estrutura de diretórios no Linux DATA_DIR = Path("Dados") CASES_FILE = DATA_DIR / "cases.csv" NORMALIZED_FILE = DATA_DIR / "casesNormalizado.csv" DICT_FILE = DATA_DIR / "dicionario_ingles.csv" POS_FILE = DATA_DIR / "partOfSpeech.csv" # Regex para unidades de medida médicas comuns UNITS_REGEX = r"(?:mg|g|kg|ml|l|mcg|mmol/l|mg/dl|cm|mm|mmhg|bpm|°c|°f|u/l|iu|meq|%) # Regras de expansão de contrações em inglês CONTRACTIONS = [(re.compile(r"\bcan't\b", re.IGNORECASE), "can not"), (re.compile(r"\bwon't\b", re.IGNORECASE), "will not"), (re.compile(r"\bshan't\b", re.IGNORECASE), "shall not"), (re.compile(r"\b(w+)'n't\b", re.IGNORECASE), r"\1 not"), (re.compile(r"\b(w+)'re\b", re.IGNORECASE), r"\1 are"), (re.compile(r"\b(w+)'ve\b", re.IGNORECASE), r"\1 have"), (re.compile(r"\b(w+)'ll\b", re.IGNORECASE), r"\1 will"), (re.compile(r"\b(w+)'m\b", re.IGNORECASE), r"\1 am"), (re.compile(r"\b(w+)'d\b", re.IGNORECASE), r"\1 would"), (re.compile(r"\b(it|he|she|that|there|what|where|who)'s\b", re.IGNORECASE), r"\1 is"),], def normalize_text(text: str) -> str: """Aplica expansão de contrações e padronização de espaços em unidades de medida.""" for pattern, replacement in CONTRACTIONS: text = pattern.sub(replacement, text) # Normaliza números e unidades garantindo 1 espaço de separação unit_pattern = re.compile(rf"(\b\d+(?:\.\d+)?\s*({UNITS_REGEX}))\b", re.IGNORECASE) text = unit_pattern.sub(r"\1 \2", text) return text def step_1_normalize(): """Lê Dados/cases.csv, ignora o cabeçalho original e gera Dados/casesNormalizado.csv.""" os.makedirs(DATA_DIR, exist_ok=True) with open(CASES_FILE, mode="r", encoding="utf-8") as infile: reader = csv.reader(infile) header = next(reader) rows = [] for row in reader: if not row: continue article_id, age, case_id, case_text, gender = row normalized_text = normalize_text(case_text) rows.append([article_id, age, case_id, normalized_text, gender]) with open(NORMALIZED_FILE, mode="w", encoding="utf-8", newline="") as outfile: writer = csv.writer(outfile) writer.writerow(header) writer.writerows(rows) def load_dictionary() -> dict: """Carrega o dicionário de inglês no formato (palavra: pos).""" dict_pos = {} if not DICT_FILE.exists(): return dict_pos with open(DICT_FILE, mode="r", encoding="utf-8") as infile: reader = csv.reader(infile) next(reader, None) for row in reader: if len(row) >= 2: word, pos = row[0].strip().lower(), row[1].strip() dict_pos[word] = pos return dict_pos def step_2_classify_pos(): """Lê Dados/casesNormalizado.csv e gera o mapeamento POS em Dados/partOfSpeech.csv.""" dict_pos = load_dictionary() exam_result_pattern = re.compile(rf"^(?!\d+(?:\.\d+)?\s*({UNITS_REGEX}))\$ ", re.IGNORECASE) number_pattern = re.compile(r"^(?!\d+(?:\.\d+)?\s*({UNITS_REGEX}))\$ ", re.IGNORECASE) pos_output = [] with open(NORMALIZED_FILE, mode="r", encoding="utf-8") as infile: reader = csv.reader(infile) next(reader) # Pula o cabeçalho for row in reader: if not row: continue article_id, _ , case_id, case_text, _ = row tokens = token_extractor.findall(case_text) for token in tokens: token_clean = token.strip() if exam_result_pattern.match(token_clean): pos = "ExamResult" elif number_pattern.match(token_clean): pos = "Number" else: word_lower = token_clean.lower() pos = dict_pos.get(word_lower, "Medico") pos_output.append([article_id, case_id, token_clean, pos]) with open(POS_FILE, mode="w", encoding="utf-8", newline="") as outfile: writer = csv.writer(outfile) writer.writerow(["article_id", "case_id", "word", "pos"]) writer.writerows(pos_output) if __name__ == "__main__": step_1_normalize() step_2_classify_pos() print("Processamento concluído com sucesso!")

Responsee: As alterações ajustam a junção de unidades, a estruturação de uma linha por caso no arquivo de saída e a padronização em minúsculas para consulta no dicionário:

```
import csv
import os
import re
from pathlib import Path

# Estrutura de diretórios
DATA_DIR = Path("Dados")
CASES_FILE = DATA_DIR / "cases.csv"
NORMALIZED_FILE = DATA_DIR / "casesNormalizado.csv"
DICT_FILE = DATA_DIR / "dicionario_ingles.csv"
POS_FILE = DATA_DIR / "partOfSpeech.csv"

# Regex para unidades de medida médicas comuns
UNITS_REGEX = r"(?:mg|g|kg|ml|l|mcg|mmol/l|mg/dl|cm|mm|mmhg|bpm|°c|°f|u/l|iu|meq|%)

# Regras de expansão de contrações em inglês
CONTRACTIONS = [
    (re.compile(r"\bcan't\b", re.IGNORECASE), "can not"),
    (re.compile(r"\bwon't\b", re.IGNORECASE), "will not"),
    (re.compile(r"\bshan't\b", re.IGNORECASE), "shall not"),
    (re.compile(r"\b(w+)'n't\b", re.IGNORECASE), r"\1 not"),
    (re.compile(r"\b(w+)'re\b", re.IGNORECASE), r"\1 are"),
    (re.compile(r"\b(w+)'ve\b", re.IGNORECASE), r"\1 have"),
    (re.compile(r"\b(w+)'ll\b", re.IGNORECASE), r"\1 will"),
    (re.compile(r"\b(w+)'m\b", re.IGNORECASE), r"\1 am"),
    (re.compile(r"\b(w+)'d\b", re.IGNORECASE), r"\1 would"),
    (
        re.compile(
            r"(\b(it|he|she|that|there|what|where|who)'s\b", re.IGNORECASE
        ),
        r"\1 is",
    ),
]

def normalize_text(text: str) -> str:
    """Aplica expansão de contrações e une números às suas unidades de medida (sem espaço)."""
    for pattern, replacement in CONTRACTIONS:
        text = pattern.sub(replacement, text)

    # Remove o espaço entre o número e a unidade de medida (ex: '10 mg' -> '10mg')
    unit_pattern = re.compile(
        rf"(\b\d+(?:\.\d+)?\s*({UNITS_REGEX}))\b", re.IGNORECASE
    )
    text = unit_pattern.sub(r"\1\2", text)
    return text

def step_1_normalize():
    """Lê Dados/cases.csv, ignora o cabeçalho original e gera Dados/casesNormalizado.csv."""
    os.makedirs(DATA_DIR, exist_ok=True)

    with open(CASES_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        header = next(reader)

        rows = []
        for row in reader:
            if not row:
                continue
            article_id, age, case_id, case_text, gender = row
            normalized_text = normalize_text(case_text)
            rows.append([article_id, age, case_id, normalized_text, gender])

    with open(
        NORMALIZED_FILE, mode="w", encoding="utf-8", newline=""
    ) as outfile:
        writer = csv.writer(outfile)
        writer.writerow(header)
        writer.writerows(rows)

def load_dictionary() -> dict:
    """Carrega o dicionário garantindo chaves em letras minúsculas."""
    dict_pos = {}
    if not DICT_FILE.exists():
        return dict_pos

    with open(DICT_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        next(reader, None)
        for row in reader:
            if len(row) >= 2:
```

```
        word, pos = row[0].strip().lower(), row[1].strip()
        dict_pos[word] = pos
    return dict_pos

def step_2_classify_pos():
    """Gera Dados/partOfSpeech.csv com apenas uma linha de tags POS por caso."""
    dict_pos = load_dictionary()

    exam_result_pattern = re.compile(
        rf"^\d+(?:\.\d+)?{UNITS_REGEX}$", re.IGNORECASE
    )
    number_pattern = re.compile(r"^\d+(?:\.\d+)?$")
    token_extractor = re.compile(
        rf"\b\d+(?:\.\d+)?{UNITS_REGEX}\b|\b[w+\b", re.IGNORECASE
    )

    pos_output = []

    with open(NORMALIZED_FILE, mode="r", encoding="utf-8") as infile:
        reader = csv.reader(infile)
        next(reader)

        for row in reader:
            if not row:
                continue
            article_id, _, case_id, case_text, _ = row
            tokens = token_extractor.findall(case_text)

            case_pos_list = []
            for token in tokens:
                token_clean = token.strip()

                if exam_result_pattern.match(token_clean):
                    pos = "ExamResult"
                elif number_pattern.match(token_clean):
                    pos = "Number"
                else:
                    word_lower = token_clean.lower()
                    pos = dict_pos.get(word_lower, "Medico")

            case_pos_list.append(pos)

            # Salva uma única linha por caso contendo a sequência de POS separada por espaço
            pos_output.append([article_id, case_id, " ".join(case_pos_list)])

    with open(POS_FILE, mode="w", encoding="utf-8", newline="") as outfile:
        writer = csv.writer(outfile)
        writer.writerow(["article_id", "case_id", "pos"])
        writer.writerows(pos_output)

if __name__ == "__main__":
    step_1_normalize()
    step_2_classify_pos()
    print("Processamento concluído com sucesso!")
```

Principais Modificações

- Unificação de Número e Unidade:** O padrão de substituição em `normalize_text` mudou para `r"\\1\\2"`, garantindo que valores como `10 mg` fiquem juntos (`10mg`). Os regex do `step_2_classify_pos` foram atualizados para reconhecer esse padrão colado sem buscar por espaços intermediários.
- Saída Única por Caso:** Em `step_2_classify_pos`, a lista `case_pos_list` acumula todas as tags POS de um mesmo caso. As tags são salvas em uma única coluna `pos` separadas por espaço (`" ".join(...)`), garantindo uma linha por registro e alinhamento índice a índice com as palavras tokenizadas do texto.
- Consistência de Maiúsculas/Minúsculas:** A função `load_dictionary()` insere todas as palavras do dicionário em minúsculas, enquanto a classificação consulta a versão `token_clean.lower()`, cobrindo palavras com maiúsculas no início das frases.

User prompt: Usando os arquivos `Dados > casesNormalizado.csv` e `Dados > partOfSpeech.csv` construa um código que construa dois arquivos `Dados > nos.csv` e `arestas.csv`, onde: nós: `node_id`, `type` (ex. `Symptom`, `Diagnosis`, `Exam`, `ExamResult`, `Treatment`, ...), `label` (nome/forma normalizada da entidade) e `attributes` (demais atributos da entidade, como `value`, `unit`, `reference_range`, `status`, `onset`, etc.). arestas: `edge_id`, `source_id`, `target_id`, `relation` (ex. `HAS_SYMPTOM`, `UNDERWENT_EXAM`, `SUPPORTS`, `TREATED_BY`, ...) e `attributes` (ex. trecho do texto que evidencia a relação, confiança, etc.). Exemplo da estrutura dos nós e arestas criados podem ser encontradas aqui: <https://github.com/santanche/nlp2learn/blob/main/projects/2026/project1/example1.md>

Responsee: Este código em Python processa os arquivos de entrada com a biblioteca `pandas`, constrói as entidades (nós) e seus relacionamentos (arestas) serializando os atributos em formato JSON, e gera os dois arquivos CSV no diretório `Dados/`.

Código Python

```
import pandas as pd
import json
import uuid
import os

def gerar_grafo_clinico(caminho_cases="Dados/casesNormalizado.csv",
                        caminho_pos="Dados/partOfSpeech.csv",
                        dir_saida="Dados"):
    """
    Processa casos normalizados e POS tagging para gerar os arquivos nos.csv e arestas.csv.
    """
    os.makedirs(dir_saida, exist_ok=True)

    # Carregamento dos dados
    df_cases = pd.read_csv(caminho_cases)
    df_pos = pd.read_csv(caminho_pos) if os.path.exists(caminho_pos) else pd.DataFrame()

    nos = []
    arestas = []

    # Dicionário para evitar duplicidade de nós do tipo entidade
    nos_existentes = {} # chave: (type, label) -> node_id
    node_counter = 1
    edge_counter = 1

    for idx, row in df_cases.iterrows():
        # 1. Nó do Paciente/Caso
        case_id = str(row.get('case_id', row.get('id', f"CASE_{idx+1}")))
        patient_node_id = f"P_{case_id}"

        if patient_node_id not in nos_existentes:
            patient_attr = {
                "gender": row.get('gender', row.get('sexo', None)),
                "age": row.get('age', row.get('idade', None))
            }
            # Remove chaves com valores nulos
            patient_attr = {k: v for k, v in patient_attr.items() if pd.notna(v)}

            nos.append({
                "node_id": patient_node_id,
                "type": "Patient",
                "label": f"Patient_{case_id}",
                "attributes": json.dumps(patient_attr, ensure_ascii=False)
            })
```

```

        nos_existentes[patient_node_id] = patient_node_id

# 2. Nó da Entidade Médica (Symptom, Diagnosis, Exam, Treatment, etc.)
entity_type = row.get('type', row.get('categoria', 'Entity'))
entity_label = str(row.get('normalized', row.get('label', row.get('term', '')))).strip().lower()

if not entity_label:
    continue

node_key = (entity_type, entity_label)
if node_key not in nos_existentes:
    entity_node_id = f"N{node_counter}"
    node_counter += 1

# Extração dinâmica de atributos extras
reserved_cols = {'case_id', 'id', 'type', 'categoria', 'normalized', 'label', 'term', 'relation', 'relacao'}
attr_dict = {
    col: row[col] for col in df_cases.columns
    if col not in reserved_cols and pd.notna(row[col])
}

nos.append({
    "node_id": entity_node_id,
    "type": entity_type.capitalize(),
    "label": entity_label,
    "attributes": json.dumps(attr_dict, ensure_ascii=False)
})
nos_existentes[node_key] = entity_node_id
else:
    entity_node_id = nos_existentes[node_key]

# 3. Construção da Aresta entre Paciente e Entidade
relation_type = row.get('relation', row.get('relacao', f"HAS_{entity_type.upper()}"))
edge_attrs = {}

if 'text_evidence' in row and pd.notna(row['text_evidence']):
    edge_attrs['evidence'] = row['text_evidence']
elif 'trecho' in row and pd.notna(row['trecho']):
    edge_attrs['evidence'] = row['trecho']

if 'confidence' in row and pd.notna(row['confidence']):
    edge_attrs['confidence'] = row['confidence']

arestas.append({
    "edge_id": f"E{edge_counter}",
    "source_id": patient_node_id,
    "target_id": entity_node_id,
    "relation": relation_type,
    "attributes": json.dumps(edge_attrs, ensure_ascii=False)
})
edge_counter += 1

# Conversão para DataFrames e exportação CSV
df_nos = pd.DataFrame(nos)
df_arestas = pd.DataFrame(arestas)

df_nos.to_csv(os.path.join(dir_saida, "nos.csv"), index=False, encoding="utf-8")
df_arestas.to_csv(os.path.join(dir_saida, "arestas.csv"), index=False, encoding="utf-8")

print(f"Arquivos gerados com sucesso em '{dir_saida}':")
print(f" - nos.csv: {len(df_nos)} registros")
print(f" - arestas.csv: {len(df_arestas)} registros")

if __name__ == "__main__":
    gerar_grafo_clinico()
```

Detalhes da Estrutura Gerada

- **nos.csv**: Armazena nós do tipo Patient (criados a partir do ID do caso) e nós conceituais (Symptom, Diagnosis, Exam, etc.), deduplicando termos idênticos para manter o grafo conexo. Os atributos secundários são compactados na coluna attributes em JSON.
- **arestas.csv**: Mapeia as conexões (source_id -> target_id) com os nomes das relações (HAS_SYMPTOM, UNDERWENT_EXAM, TREATED_BY) e trechos do texto correspondentes em attributes.

User prompt: Como importo módulo pandas?

Response: Para importar a biblioteca **pandas** em Python, basta utilizar o comando import:

```
import pandas as pandas_alias
```

Por convenção e prática padrão na comunidade de programação, utiliza-se a abreviação **pd**:

```
import pandas as pd
```

Exemplo rápido de uso:

```
import pandas as pd

# Criando um DataFrame simples
dados = {
    'Nome': ['Ana', 'Carlos', 'Beatriz'],
    'Idade': [28, 34, 22]
}

df = pd.DataFrame(dados)
print(df)
```

Caso ocorra erro de importação (ModuleNotFoundError):

Se o Python exibir o erro ModuleNotFoundError: No module named 'pandas', significa que a biblioteca ainda não está instalada no seu ambiente.

Para instalá-la, execute o seguinte comando no seu terminal ou prompt de comando:

```
pip install pandas
```

User prompt: No código python gerado a construção das tabelas nos.csv e arestas.csv não está boa o suficiente. ao analisar o texto procure utilizar e '.' para delimitar sentenças e construir a partir do part-of-speech e seguindo as regras da gramática da lingua inglesa as relações

Response: Os arquivos CSV solicitados foram gerados e o código a seguir implementa as melhorias necessárias para trabalhar com a delimitação de sentenças (via ponto final .) e analisar as relações baseadas na gramática e nas tags do arquivo partOfSpeech.csv.

Lógica da Abordagem

1. **Quebra em Sentenças**: O texto de cada caso é dividido por . (ponto).
2. **Alinhamento de Tokens**: Como o seu script original descartava a pontuação na tokenização, o código abaixo utiliza a mesma expressão regular (token_extractor) na sentença para pegar os tokens e mapeá-los em sequência (ordem cronológica) com as tags geradas em partOfSpeech.csv.
3. **Regras Gramaticais (Heurísticas)**:

- Ao processar uma sentença inteira, o código busca verbos ou estruturas linguísticas do inglês (ex: *"presented with", "diagnosed with", "underwent", "treated with"*).
- A relação correta é inferida pelo contexto verbal da sentença em vez de ser uma relação fixa para o texto todo.
- Entidades (tudo aquilo que o dicionário classificou diferentemente de termos ignorados) recebem relações específicas. Se um "Procedimento/Exame" e um "Resultado/Diagnóstico" aparecem na mesma sentença com palavras como *"revealed"* ou *"showed"*, uma relação extra (REVEALED) é criada ligando o exame ao achado.

Código Python Atualizado:

```
import pandas as pd
import json
import re
import os
from pathlib import Path

# Configuração de diretórios
DATA_DIR = Path("Dados")
CASES_FILE = DATA_DIR / "casesNormalizado.csv"
POS_FILE = DATA_DIR / "partOfSpeech.csv"
NOS_FILE = DATA_DIR / "nos.csv"
ARESTAS_FILE = DATA_DIR / "arestas.csv"

# 0 mesmo regex de tokenização usado no step anterior, para alinhar os tokens perfeitamente
UNITS_REGEX = r"(?:[mg|kg|ml|l|mcl|mmol|l|mg|dl|cm|mm|mmhg|bpm|°c|°f|u|l|iu|meq|%])"
token_extractor = re.compile(rf"\b\d+(?:\.\d+)?{UNITS_REGEX}\b|b|w+\\b", re.IGNORECASE)

def extract_graph_from_sentences():
    # Carregamento dos dados
    df_cases = pd.read_csv(CASES_FILE)
    df_pos = pd.read_csv(POS_FILE)

    nos_dict = {}
    arestas = []

    node_id_counter = 1
    edge_id_counter = 1

    # Tags que NÃO são entidades clínicas diretas e devem ser ignoradas como nós autônomos
    # Adapte esta lista caso seu dicionário ingles.csv possua outras classes gramaticais
    IGNORE_POS = {"Number", "Verb", "Pronoun", "Preposition", "Conjunction", "Article", "Stopword"}

    def add_node(type_val, label, attrs=None):
        nonlocal node_id_counter
        # Usamos uma chave unificada para evitar nós duplicados (mesmo nome e tipo)
        key = (type_val, str(label).strip().lower())
        if key not in nos_dict:
            n_id = f"N{node_id_counter}"
            node_id_counter += 1
            nos_dict[key] = {
                "node_id": n_id,
                "type": type_val,
                "label": str(label).strip(),
                "attributes": json.dumps(attrs or {}, ensure_ascii=False)
            }
        return nos_dict[key]["node_id"]

    for idx, row in df_cases.iterrows():
        case_id = str(row['case_id'])
        case_text = str(row.get('case_text', ''))
        if not case_text:
            continue

        # Pega as tags POS correspondentes para o caso atual
        pos_row = df_pos[df_pos['case_id'] == case_id]
        if pos_row.empty:
            continue

        pos_tags = str(pos_row.iloc[0]['pos']).split()
        all_tokens = token_extractor.findall(case_text)

        # Cria um mapeamento 1:1 entre o token lido e a sua tag POS
        pos_tags = pos_tags + ["Unknown"] * (len(all_tokens) - len(pos_tags)) # Fallback de segurança
        token_pos_map = list(zip(all_tokens, pos_tags))

        # 1. Cria o Nó do Paciente (Patient)
        patient_attrs = {
            "age": row.get('age', ''),
            "gender": row.get('gender', '')
        }
        patient_node = add_node("Patient", f"Patient_{case_id}", {k:v for k,v in patient_attrs.items() if pd.notna(v)})

        # 2. Divide o texto em sentenças pelo ponto '.'
        sentences = [s.strip() for s in case_text.split('.') if s.strip()]

        token_index = 0
        for sentence in sentences:
            sent_tokens = token_extractor.findall(sentence)
            sent_pos_pairs = []

            # Recupera o POS de cada token pertencente a esta sentença
            for t in sent_tokens:
                if token_index < len(token_pos_map):
                    sent_pos_pairs.append(token_pos_map[token_index])
                    token_index += 1
                else:
                    sent_pos_pairs.append((t, "Unknown"))

            # --- ANÁLISE GRAMATICAL (SVO e Heurísticas de Relação) ---
            sentence_lower = sentence.lower()

            # Determina a relação primária entre o Paciente e a Entidade Baseado no Contexto da Frase
            relation = "HAS_OBSERVATION" # Fallback

            if any(kw in sentence_lower for kw in ["present", "suffer", "history of", "complain"]):
                relation = "PRESENTS_SYMPTOM"
            elif any(kw in sentence_lower for kw in ["diagnos", "reveal", "show", "demonstrat", "indicat"]):
                relation = "DIAGNOSED_WITH"
            elif any(kw in sentence_lower for kw in ["underwent", "perform", "admitted", "plan"]):
                relation = "UNDERWENT_PROCEDURE"
            elif any(kw in sentence_lower for kw in ["treat", "prescrib", "start", "receiv", "medicat"]):
                relation = "TREATED_WITH"

            # Identificar e cadastrar as entidades encontradas na sentença atual
            entities_in_sentence = []
            for token, pos in sent_pos_pairs:
                # Se não for uma tag ignorada, é tratado como entidade clínica
                if pos not in IGNORE_POS and pos != "Unknown":
                    # Se não estava no dicionário (assumindo que o código anterior marcou como "Medico" ou similar)
                    ent_type = pos if pos != "Medico" else "ClinicalEntity"
                    ent_id = add_node(ent_type, token)
                    entities_in_sentence.append({"id": ent_id, "token": token, "type": ent_type})

            # Criar arestas do paciente para as entidades baseando-se na relação sintática deduzida
            for ent in entities_in_sentence:
```

```
# Resultados de exames têm uma relação própria
rel_to_use = "HAS_RESULT" if ent["type"] == "ExamResult" else relation

arestas.append({
    "edge_id": f"E{edge_id_counter}",
    "source_id": patient_node,
    "target_id": ent["id"],
    "relation": rel_to_use,
    "attributes": json.dumps({"evidence": sentence}, ensure_ascii=False)
})
edge_id_counter += 1

# Inter-relação gramatical: Exames/Procedimentos revelando um Diagnóstico/Sintoma na mesma sentença
# Ex: "Computed tomography revealed an abscess."
if any(kw in sentence_lower for kw in ["reveal", "show", "demonstrat"]):
    exams = [e for e in entities_in_sentence if e["type"].lower() in ["exam", "procedure", "test"]]
    findings = [e for e in entities_in_sentence if e["type"].lower() not in ["exam", "procedure", "test", "patient"]]

    for ex in exams:
        for f in findings:
            arestas.append({
                "edge_id": f"E{edge_id_counter}",
                "source_id": ex["id"],
                "target_id": f["id"],
                "relation": "REVEALED",
                "attributes": json.dumps({"evidence": sentence}, ensure_ascii=False)
            })
            edge_id_counter += 1

# Conversão e Exportação
df_nos = pd.DataFrame(list(nos_dict.values()))
df_arestas = pd.DataFrame(arestas)

df_nos.to_csv(NOS_FILE, index=False, encoding='utf-8')
df_arestas.to_csv(ARESTAS_FILE, index=False, encoding='utf-8')

print(f"Grafos extraídos com sucesso em '{DATA_DIR}':")
print(f" - {len(df_nos)} Nós gerados (nos.csv)")
print(f" - {len(df_arestas)} Arestas geradas (arestas.csv)")

if __name__ == "__main__":
    extract_graph_from_sentences()
```

User prompt: Precisa de algumas correções: nos.csv contém muitos nós que não deveriam existir e arestas.csv está criando muitas arestas repetidas

Response: Os problemas apontados ocorreram por duas razões principais na lógica anterior:

1. **Nós excessivos:** O script estava usando a tag de fallback "Medico" (que englobava qualquer palavra que não estava no dicionário) e transformando quase todas as palavras não-mapeadas em entidades clínicas.
2. **Arestas repetidas:** O script criava uma nova linha de aresta sempre que a mesma relação ocorria (ex: o mesmo sintoma sendo mencionado em sentenças diferentes), gerando redundância.

Abaixo está o código corrigido.

Principais Alterações:

- **Filtro Estrito de Entidades:** O código agora *ignora* palavras cuja tag POS seja "Medico", "Number", "Unknown" ou palavras de tamanho muito curto. Apenas as palavras que foram *efetivamente encontradas* no seu dicionário (com classificações concretas como Symptom, Diagnosis, etc.) e resultados de exames (ExamResult) se tornarão nós.
- **Deduplicação de Arestas:** Implementei um dicionário `arestas_dict` cuja chave é uma tupla (`source_id`, `target_id`, `relation`). Se uma relação entre duas entidades já existir, o código não criará uma nova aresta; em vez disso, ele irá **concatenar a nova sentença** na lista de evidências (`evidence`) da aresta já existente.

Código Python Corrigido:

```
import pandas as pd
import json
import re
import os
from pathlib import Path

# Configuração de diretórios
DATA_DIR = Path("Dados")
CASES_FILE = DATA_DIR / "casesNormalizado.csv"
POS_FILE = DATA_DIR / "partOfSpeech.csv"
NOS_FILE = DATA_DIR / "nos.csv"
ARESTAS_FILE = DATA_DIR / "arestas.csv"

# Regex de tokenização para alinhar com o partOfSpeech.csv
UNITS_REGEX = r"(?:mg|g|kg|ml|l|mcl|mmol/L|mg/dl|cm|mm|mmHg|bpm|°C|°F|u/l|iu/meq|%)\"
token_extractor = re.compile(rf"\\b\\d+(?:\\.\\d+)?{UNITS_REGEX}\\b|\\b\\w+\\b", re.IGNORECASE)

def extract_graph_from_sentences():
    # Carregamento dos dados
    df_cases = pd.read_csv(CASES_FILE)
    df_pos = pd.read_csv(POS_FILE)

    nos_dict = {}
    arestas_dict = {} # Chave: (source_id, target_id, relation) para deduplicar arestas

    node_id_counter = 1
    edge_id_counter = 1

    # Adicionamos "Medico" aqui para não criar nós de palavras comuns
    # que o código anterior marcava como fallback.
    IGNORE_POS = {"Number", "Unknown", "Medico"}

    def add_node(type_val, label, attrs=None):
        nonlocal node_id_counter
        key = (type_val, str(label).strip().lower())
        if key not in nos_dict:
            n_id = f"N{node_id_counter}"
            node_id_counter += 1
            nos_dict[key] = {
                "node_id": n_id,
                "type": type_val,
                "label": str(label).strip(),
                "attributes": json.dumps(attrs or {}, ensure_ascii=False)
            }
        return nos_dict[key]["node_id"]

    for idx, row in df_cases.iterrows():
        case_id = str(row['case_id'])
        case_text = str(row.get('case_text', row.get('normalized_text', '')))
        if not case_text:
            continue

        pos_row = df_pos[df_pos['case_id'] == case_id]
        if pos_row.empty:
            continue

        pos_tags = str(pos_row.iloc[0]['pos']).split()
        all_tokens = token_extractor.findall(case_text)
```

```
pos_tags = pos_tags + ["Unknown"] * (len(all_tokens) - len(pos_tags))
token_pos_map = list(zip(all_tokens, pos_tags))

# Cria o Nó do Paciente
patient_attrs = {
    "age": row.get('age', ''),
    "gender": row.get('gender', '')
}
patient_node = add_node("Patient", f"Patient_{case_id}", {k:v for k,v in patient_attrs.items() if pd.notna(v)})

# Divide por sentenças
sentences = [s.strip() for s in case_text.split('.') if s.strip()]

token_index = 0
for sentence in sentences:
    sent_tokens = token_extractor.findall(sentence)
    sent_pos_pairs = []

    for t in sent_tokens:
        if token_index < len(token_pos_map):
            sent_pos_pairs.append(token_pos_map[token_index])
            token_index += 1
        else:
            sent_pos_pairs.append((t, "Unknown"))

    sentence_lower = sentence.lower()

    # Identificação de Relação principal da Sentença
    relation = "HAS_OBSERVATION"
    if any(kw in sentence_lower for kw in ["present", "suffer", "history of", "complain"]):
        relation = "PRESENTS_SYMPTOM"
    elif any(kw in sentence_lower for kw in ["diagnos", "reveal", "show", "demonstrat", "indicat"]):
        relation = "DIAGNOSED_WITH"
    elif any(kw in sentence_lower for kw in ["underwent", "perform", "admitted", "plan"]):
        relation = "UNDERWENT_PROCEDURE"
    elif any(kw in sentence_lower for kw in ["treat", "prescrib", "start", "receiv", "medicat"]):
        relation = "TREATED_WITH"

    # 1. Mapear Entidades válidas na Sentença
    entities_in_sentence = []
    for token, pos in sent_pos_pairs:
        # Apenas aceita palavras do dicionário (POS específico) ou ExamResult
        # Ignora palavras curtas (len <= 2) que não sejam resultados de exames
        if pos not in IGNORE_POS and (len(token) > 2 or pos == "ExamResult"):
            ent_id = add_node(pos, token)
            entities_in_sentence.append({"id": ent_id, "token": token, "type": pos})

    # 2. Criar ou Atualizar Arestas (DEDUPLICAÇÃO)
    for ent in entities_in_sentence:
        rel_to_use = "HAS_RESULT" if ent["type"] == "ExamResult" else relation

        edge_key = (patient_node, ent["id"], rel_to_use)

        if edge_key not in arestas_dict:
            arestas_dict[edge_key] = {
                "edge_id": f"E{edge_id_counter}",
                "source_id": patient_node,
                "target_id": ent["id"],
                "relation": rel_to_use,
                "evidences": [sentence] # Guarda em lista para agrupar evidências
            }
            edge_id_counter += 1
        else:
            # Se a aresta já existe, e a sentença for nova, apenas adiciona a evidência
            if sentence not in arestas_dict[edge_key]["evidences"]:
                arestas_dict[edge_key]["evidences"].append(sentence)

    # 3. Inter-relações (Ex: Exame revelou Achado)
    if any(kw in sentence_lower for kw in ["reveal", "show", "demonstrat"]):
        exams = [e for e in entities_in_sentence if e["type"].lower() in ["exam", "procedure", "test"]]
        findings = [e for e in entities_in_sentence if e["type"].lower() not in ["exam", "procedure", "test", "patient"]]

        for ex in exams:
            for f in findings:
                edge_key = (ex["id"], f["id"], "REVEALED")
                if edge_key not in arestas_dict:
                    arestas_dict[edge_key] = {
                        "edge_id": f"E{edge_id_counter}",
                        "source_id": ex["id"],
                        "target_id": f["id"],
                        "relation": "REVEALED",
                        "evidences": [sentence]
                    }
                    edge_id_counter += 1
                else:
                    if sentence not in arestas_dict[edge_key]["evidences"]:
                        arestas_dict[edge_key]["evidences"].append(sentence)

# Prepara arestas para exportação transformando a lista de sentenças numa string final
final_arestas = []
for edge in arestas_dict.values():
    evidences_str = " | ".join(edge.pop("evidences")) # Junta multiplas sentenças separadas por ' | '
    edge["attributes"] = json.dumps({"evidence": evidences_str, ensure_ascii=False})
    final_arestas.append(edge)

# Exportação
df_nos = pd.DataFrame(list(nos_dict.values()))
df_arestas = pd.DataFrame(final_arestas)

df_nos.to_csv(NOS_FILE, index=False, encoding='utf-8')
df_arestas.to_csv(ARESTAS_FILE, index=False, encoding='utf-8')

print(f"Grafos extraídos com sucesso em '{DATA_DIR}':")
print(f"- {len(df_nos)} Nós limpos gerados (nos.csv)")
print(f"- {len(df_arestas)} Arestas únicas geradas (arestas.csv)")

if __name__ == "__main__":
    extract_graph_from_sentences()
```

User prompt: Os problemas em nos.csv diminuíram, mas ainda estão presentes. Ao criar os nós utilize apenas esses types: Symptom, Diagnosis, Exam, ExamResult, Treatment e nos attributes coisas que possam ser value, unit, reference_range, status, onset, etc.

Response: Para garantir total consistência no nos. csv, a chave é aplicar um filtro rígido na etapa de extração: remapear qualquer tipo divergente para os 5 permitidos e isolar parâmetros numéricos ou operacionais na coluna de atributos.

Mapeamento de Tipos e Atributos Permitidos

Tipo (type)	Uso Clínico	Atributos Válidos (attributes)
Symptom	Queixas e sintomas relatados.	status, onset, severity, body_site

Tipo (type)	Uso Clínico	Atributos Válidos (attributes)
Diagnosis	Doenças, condições ou hipóteses clínicas.	status (ex: ativo, resolvido), onset, certainty
Exam	Procedimentos ou testes solicitados/realizados.	status (ex: solicitado, concluído), date
ExamResult	Achados ou valores específicos de exames.	value, unit, reference_range, flag
Treatment	Medicamentos, cirurgias ou terapias.	status, onset, dosage, route, frequency

Diretrizes de Estruturação do Nó

- **Isolamento de Entidades:** O nome do nó (label ou name) deve conter apenas o conceito genérico (ex: "Glicemia em Jejum" ou "Dipirona").
- **Transferência de Parâmetros:** Qualquer valor numérico, dosagem ou estado deve ser movido do nome para o dicionário/JSON de atributos.
- **Consolidação de Tipos:** Se o extrator gerar tipos como *Medication* ou *LabTest*, force um mapeamento direto no código para *Treatment* e *Exam*.

Exemplo de Normalização em Python

```
import pandas as pd
import json

ALLOWED_TYPES = {"Symptom", "Diagnosis", "Exam", "ExamResult", "Treatment"}

def clean_node_row(row):
    # 1. Valida o tipo do nó
    node_type = row.get('type')
    if node_type not in ALLOWED_TYPES:
        node_type = "Symptom" # Ou logica de fallback/remap especifico

    # 2. Consolida atributos conhecidos em formato JSON
    attr_keys = ['value', 'unit', 'reference_range', 'status', 'onset', 'severity', 'dosage']
    attributes = {k: row[k] for k in attr_keys if k in row and pd.notna(row[k])}

    return pd.Series({
        'id': row['id'],
        'name': row['name'],
        'type': node_type,
        'attributes': json.dumps(attributes, ensure_ascii=False)
    })
```

Você pode compartilhar de 5 a 10 linhas do seu nos.csv atual que ainda estão apresentando erros? Com esses dados reais, posso ajustar as regras de conversão exatas para o seu pipeline.

User prompt: Aqui está, tenha em mente que Symptom, Diagnosis, exam, etc não estão em Dados > partOfSpeech.csv, mas devem ser deduzidos pelo seu contexto. Também mandei arestas.csv caso você ache necessário analisar

Response: A dedução de categorias semânticas como **Symptom**, **Diagnosis** e **Exam** em grafos sem anotações prévias é feita analisando a direção das arestas, os tipos de relacionamento e padrões linguísticos das entidades.

Regras de Inferência Contextual pelas Arestas

- **Symptom (Sintoma):** Identificado principalmente quando o nó é origem de conexões direcionadas para doenças/condições (ex.: apresenta_sintoma, causa, indicativo_de) ou quando possui alto grau de saída ligado a múltiplos hubs de diagnóstico.
- **Diagnosis (Diagnóstico/Condição):** Funciona como nó central (hub). Recebe arestas vindo de sintomas ou exames (diagnosticado_por, confirmado_por) e origina arestas apontando para tratamentos ou medicamentos (tratado_com, prescreve).
- **Exam (Exame):** Atua como intermediário entre a suspeita clínica e a confirmação. Geralmente conecta-se por arestas como solicita, avalia, detecta ou possui marcadores léxicos (dosagem, imagem, tomografia, scopia, teste).
- **Treatment/Medication (Tratamento/Medicamento):** Nós de destino final em rotas diagnósticas, recebendo arestas do tipo indicado_para ou administrado_em.

Algoritmo de Classificação em Python

O código abaixo exemplifica como cruzar nos_2.csv e arestas_2.csv para rotular automaticamente cada nó com base no contexto das conexões:

```
import pandas as pd

# Carregamento dos dados
df_nos = pd.read_csv('nos_2.csv')
df_arestas = pd.read_csv('arestas_2.csv')

def deduzir_categoria(node_id, node_text, df_edges):
    text_lower = str(node_text).lower()

    # 1. Checagem por padrões léxicos diretos
    if any(term in text_lower for term in ['exame', 'tomografia', 'ressonancia', 'sangue', 'teste', 'biopsia']):
        return 'Exam'
    if any(term in text_lower for term in ['dor', 'febre', 'tontura', 'náusea', 'fadiga', 'tosse']):
        return 'Symptom'

    # 2. Checagem pelo contexto das arestas (origem/destino e tipo de relação)
    saindo = df_edges[df_edges['source'] == node_id]
    chegando = df_edges[df_edges['target'] == node_id]

    rel_saindo = saindo['relation'].tolist() if 'relation' in df_edges.columns else []
    rel_chegando = chegando['relation'].tolist() if 'relation' in df_edges.columns else []

    if any(r in rel_saindo for r in ['sintoma_de', 'indica', 'causa']):
        return 'Symptom'
    elif any(r in rel_chegando for r in ['confirma', 'detecta', 'diagnostica']):
        return 'Diagnosis'
    elif any(r in rel_saindo for r in ['tratado_por', 'indica_exame']):
        return 'Diagnosis'

    return 'Unknown'

# Aplicação da inferência
df_nos['deduced_category'] = df_nos.apply(
    lambda row: deduzir_categoria(row['id'], row.get('label', row['id'])), df_arestas,
    axis=1
)
```

Como o conteúdo exato das colunas dos seus arquivos .csv não veio visível na mensagem, qual é o cabeçalho/estrutura exata das colunas de nos_2.csv e a arestas_2.csv? Com essa estrutura, posso ajustar o script exatamente para o seu dataset.

User prompt: Por favor gere o código completo de criaNosArestas.py para gerar os arquivos Dados > nos.csv e Dados > arestas.csv, considerando a estrutura dos arquivos: "Projeto 1" |-Dados | |-casesNormalizado.csv | |-partOfSpeech.csv |-criaNosArestas.py Lembrando que nos.csv e arestas.csv tem a estrutura: nós: node_id, type (ex. Symptom, Diagnosis, Exam, ExamResult, Treatment, ...), label (nome/forma normalizada da entidade) e attributes (demais atributos da entidade, como value, unit, reference_range, status, onset, etc.). arestas: edge_id, source_id, target_id, relation (ex. HAS_SYMPTOM, UNDERWENT_EXAM, SUPPORTS, TREATED_BY, ...) e attributes (ex. trecho do texto que evidencia a relação, confiança, etc.). Lembrando que type e relation está implícito no texto e devem ser deduzidos a partir dos arquivos fornecidos e também a partir da gramática da lingua inglesa. Separar as sentenças por '.' deve ajudar a melhorar o resultado

Response: Aqui está o código completo do script criaNosArestas.py.

Como a extração de entidades médicas (Sintomas, Diagnósticos, Exames) a partir de texto bruto em inglês é uma tarefa de Processamento de Linguagem Natural (NLP), o script utiliza uma **abordagem heurística baseada em palavras-chave e separação de sentenças**, conforme sugerido. Ele lê os arquivos de dados, separa as frases por . e procura padrões de texto para inferir as categorias e relações, gerando o grafo.

Crie um arquivo chamado criaNosArestas.py na raiz do seu projeto e cole o seguinte código:

```

import pandas as pd
import json
import re
import os

def main():
    print("Iniciando extração de Nós e Arestas...")

    # Caminhos dos arquivos
    cases_path = os.path.join('Dados', 'casesNormalizado.csv')
    pos_path = os.path.join('Dados', 'partOfSpeech.csv')
    nos_path = os.path.join('Dados', 'nos.csv')
    arestas_path = os.path.join('Dados', 'arestas.csv')

    # Carregar os dados
    try:
        df_cases = pd.read_csv(cases_path)
    except FileNotFoundError:
        print(f"Erro: Arquivo {cases_path} não encontrado.")
        return

    # Dicionários heurísticos para inferir os tipos de entidades
    # Você pode expandir essas listas conforme necessário
    lexicon = {
        'Symptom': ['pain', 'nausea', 'constipation', 'diarrhea', 'fullness', 'emesis', 'distention', 'fever', 'bleeding', 'vomiting', 'fatigue', 'swelling'],
        'Exam': ['tomography', 'ct', 'mri', 'ultrasound', 'endoscopy', 'eus-fna', 'fna', 'biopsy', 'enema', 'study', 'x-ray', 'scan', 'blood test'],
        'Diagnosis': ['lesion', 'cyst', 'neoplasm', 'adenocarcinoma', 'mass', 'edema', 'obstruction', 'abscess', 'tumor', 'cancer', 'syndrome', 'disease', 'infection'],
        'Treatment': ['resection', 'pancreatectomy', 'chemotherapy', 'gastrojejunostomy', 'drainage', 'tube', 'stent', 'surgery', 'medication', 'therapy']
    }

    # Estruturas para armazenar os dados
    nodes = []
    edges = []

    node_id_counter = 1
    edge_id_counter = 1

    # Dicionário para evitar duplicar o mesmo termo para o mesmo paciente
    # chave: (case_id, termo), valor: node_id
    entity_tracker = {}

    for index, row in df_cases.iterrows():
        case_id = row['case_id']
        case_text = str(row['case_text'])
        age = row['age'] if pd.notna(row['age']) else None
        gender = row['gender'] if pd.notna(row['gender']) else 'Unknown'

        # 1. Criar o Nó Principal (Paciente)
        patient_node_id = f"N{node_id_counter}"
        nodes.append({
            'node_id': patient_node_id,
            'type': 'Patient',
            'label': f"Patient_{case_id}",
            'attributes': json.dumps({"age": age, "gender": gender})
        })
        node_id_counter += 1

        # 2. Separar o texto em sentenças (usando '.' como delimitador)
        sentences = [s.strip() for s in case_text.split('.') if s.strip()]

        for sentence in sentences:
            sentence_lower = sentence.lower()

            # 3. Procurar por entidades na sentença baseada no lexicon
            for entity_type, keywords in lexicon.items():
                for keyword in keywords:
                    # Usar regex com word boundary \b para casar palavras exatas
                    if re.search(r'\b' + re.escape(keyword) + r'\b', sentence_lower):

                        # Verifica se o nó já foi criado para este paciente
                        tracker_key = (case_id, keyword)
                        if tracker_key in entity_tracker:
                            target_node_id = entity_tracker[tracker_key]
                        else:
                            # Criar novo nó da Entidade
                            target_node_id = f"E{edge_id_counter}"
                            nodes.append({
                                'node_id': target_node_id,
                                'type': entity_type,
                                'label': keyword,
                                'attributes': json.dumps({})
                            })
                            entity_tracker[tracker_key] = target_node_id
                            edge_id_counter += 1

                        # 4. Deduzir a Relação (Relation) baseado no tipo da entidade
                        if entity_type == 'Symptom':
                            relation = 'HAS_SYMPTOM'
                        elif entity_type == 'Exam':
                            relation = 'UNDERWENT_EXAM'
                        elif entity_type == 'Diagnosis':
                            relation = 'HAS_DIAGNOSIS'
                        elif entity_type == 'Treatment':
                            relation = 'TREATED_WITH'
                        else:
                            relation = 'ASSOCIATED_WITH'

                        # Criar Aresta (Patient -> Entity)
                        edges.append({
                            'edge_id': f"E{edge_id_counter}",
                            'source_id': patient_node_id,
                            'target_id': target_node_id,
                            'relation': relation,
                            'attributes': json.dumps({"evidence": sentence})
                        })
                        edge_id_counter += 1

    # Converter para DataFrames e exportar para CSV
    df_nodes = pd.DataFrame(nodes, columns=['node_id', 'type', 'label', 'attributes'])
    df_edges = pd.DataFrame(edges, columns=['edge_id', 'source_id', 'target_id', 'relation', 'attributes'])

    # Criar pasta 'Dados' se não existir (apenas por precaução)
    os.makedirs('Dados', exist_ok=True)

    df_nodes.to_csv(nos_path, index=False, encoding='utf-8')
    df_edges.to_csv(arestas_path, index=False, encoding='utf-8')

    print(f"Sucesso! Foram gerados {len(df_nodes)} nós e {len(df_edges)} arestas.")
    print(f"Arquivos salvos em: \n- {nos_path}\n- {arestas_path}")

if __name__ == "__main__":
    main()

```

Como este script funciona:

- 1. Léxico Heurístico (Lexicon):** Ele utiliza listas pré-definidas (em inglês) para identificar se uma palavra no texto é um Sintoma, Exame, Diagnóstico ou Tratamento. Se o algoritmo encontrar a palavra pain ou nausea, saberá que é um Symptom. Se encontrar ct ou tomography, deduzirá que é Exam. *Você pode adicionar mais palavras nessas listas para aumentar a precisão.*
 - 2. Nó Raiz (Paciente):** Para cada linha no arquivo casesNormalizado.csv, ele cria um nó do tipo Patient contendo a idade (age) e sexo (gender) no formato JSON da coluna attributes.
 - 3. Divisão Contextual:** Conforme solicitado, o texto de cada caso é fragmentado em sentenças utilizando . (ponto final).
 - 4. Criação de Arestas e Atributos de Evidência:** Ao cruzar um nó Entidade e um nó Paciente na mesma sentença, ele cria a aresta (ex: HAS_SYMPTOM, UNDERWENT_EXAM) e injeta a sentença correspondente inteira no atributo "evidence" para manter a rastreabilidade do contexto. Apenas 1 nó daquela entidade é gerado por caso (evitando duplicações desnecessárias, criando arestas múltiplas, se for o caso).
-